



C2000™ CLA Software Development Guide

1.0

Copyright © 2020, Texas Instruments Incorporated

Online HTML version available [here](#)



C2000™ CLA 软件开发指南1.0

Copyright © 2020, Texas Instruments Incorporated

Online HTML version available [here](#)

CONTENTS

目录

1 Overview 1

2 Getting Started 3

2.1 Workshops 3

2.2 Development Tools 4

2.2.1 Compiler and Assembly Tools 4

2.2.2 MathWorks Embedded Coder Support 4

2.3 Examples 4

2.4 FAQs and Debugging Tips 5

2.5 System Use Cases 5

3 Identifying Device-Specific CLA Features 6

3.1 CLA Types 6

3.2 Device-Specific Features 8

4 Frequently Asked Questions 9

4.1 Is the CLA independent from the C28x CPU? 9

4.2 Can the C28x and CLA be synchronized? 9

4.3 Is the CLA programmable? 9

4.4 Is there a C compiler for the CLA? 9

4.5 How is data shared between the CLA and C28x? 10

4.6 How are data types different on C28x and CLA? 10

4.7 What can trigger a task (also referred to as an ISR)? 13

4.8 Is there support for nesting tasks? 14

4.9 Is there a limitation on the code size for tasks? 14

4.10 Which peripheral registers can the CLA access directly? 15

4.11 Can the CLA send an interrupt to the C28x? 16

4.12 Does the CLA have access to all memory blocks? 16

4.13 If C28x and CLA can access the same device resource, which has priority? 17

4.14 How can I measure the duration of a task? 17

4.15 Can the C28x terminate a CLA task? 18

1 概述 1 2 入门 3 2.1 研讨会.3 2.2 2.2 开发工具.4 2.2.1 编译器与汇编工具.4 2.2.2 MathWorks Embedded Coder 支持.4 2.3 示例.4 2.4 常见问题与调试技巧.5 2.5 系统用例.5 3 识别设备特定的 CLA 特性 6 3.1 CLA 类型.6 3.2 设备特定功能.8 4 常见问题解答 9 4.1 CLA 是否独立于 C28x CPU? .9 4.2 C28x 与 CLA 能否同步? .9 4.3 CLA 是否可编程? .9 4.4 是否有针对 CLA 的 C 编译器? .9 4.5 CLA 与 C28x 之间如何共享数据? .10 4.6 C28x 和 CLA 上的数据类型有何不同? .10 4.7 什么可以触发任务（也称为 中断服务例程（ISR））? .13 4.8 是否支持任务嵌套? .14 4.9 任务的代码大小是否有限制? .14 4.10 CLA 可以直接访问哪些外设寄存器? .15 4.11 CLA 能否向 C28x 发送中断? .16 4.12 CLA 是否可以访问所有存储块? .16 4.13 如果 C28x 和 CLA 可以访问相同的设备资源，哪个具有优先级? ... 17 4.14 我如何测量任务的持续时间? .17 4.15 C28x 能终止 CLA 任务吗? .18

4.16	What does the “Symbol, X, referenced in a.obj, assumes that data is blocked but is accessing non-blocked data in b.obj. Runtime failures may result” warning from the Linker mean?	18
5	Debugging Tips	19
5.1	Help! Why doesn’t my CLA code work?	20
5.2	My CLA task never starts.	20
5.3	Why can’t I force a task using software (IACK)?	21
5.4	Why does my Code Composer Studio (CCS) GEL file enable the CLA clock on reset?	21
5.5	Can I start a task from the debug window?	21
5.6	After a “run”, why did the CLA begin executing another task instead of stopping at MSTOP?	21
5.7	Why are the variables in the CLA code not updating as expected?	22
6	Comparison to C28x+FPU	23
6.1	Are benchmarks for FPU div, sin, cos...etc. the same?	23
6.2	Is the CLA floating-point multiply faster?	24
6.3	What are the main differences?	24
7	Support	26
8	IMPORTANT NOTICE AND DISCLAIMER	27
9	Changelog	28

4.16	链接器中 “Symbol, X, referenced in a.obj, assumes that data is blocked but is accessing non-blocked data in b.obj. Runtime failures may result” （在 a.obj 中引用的符号 X 假定数据被屏蔽，但正在访问 b.obj 中的非屏蔽数据。可能导致运行时故障）警告是什么意思？ .185
调试提示	195.1 求助！为什么我的 CLA 代码不起作用？ .205.2 我的 CLA 任务从未启动。 .205.3 为什么我不能通过软件强制启动任务 (IACK)? .215.4 为什么我的 Code Composer Studio（代码作曲家工作室）GEL 文件在复位时启用 CLA 时钟？ .215.5 我可以从调试窗口启动任务吗？ .215.6 在一次“运行”后，为什么 CLA 在 MSTOP 处没有停止而开始执行另一个任务？ .215.7 为什么 CLA 代码中的变量没有按预期更新？
226	与 C28x 的比较 +FPU
236.1	FPU 的 div、sin、cos 等基准测试是相同的吗？ .236.2 CLA 的浮点乘法更快吗？ .246.3 主要差异是什么？ .247
支持	268
重要通知和免责声明	279
更改日志	28

OVERVIEW

The Control Law Accelerator (CLA) available on C2000™ MCUs is a fully-programmable independent 32-bit floating-point hardware accelerator that is designed for math intensive computations. The CLA can offer a significant boost to the performance of typical math functions that are commonly found in control algorithms. The CLA is designed to execute real-time control algorithms in parallel with the C28x CPU, effectively doubling the computational performance. This makes the CLA perfect for managing low-level control loops with higher cycle performance improvements over the C28x CPU. The CLA is also designed for fast interrupt response and has direct access to peripherals such as ePWM, HRPWM, eCAP, eQEP, ADC, DAC, CMPSS, PGA, SDFM, SPI, LIN, FSI, PMBus, CLB, and GPIO.

The CLA increases the overall compute bandwidth of the C2000 MCU and frees up the CPU for other tasks such as communications, systems, and diagnostics. Fig. 1.1 illustrates the key benefits of the CLA:

1. Reduced interrupt latency
2. Reduced control loop delay
3. Reduced C28x CPU bandwidth usage

概述

The Control Law Accelerator (CLA) available on C2000™ MCUs 是一个完全可编程的独立 32 位浮点数 硬件加速器，专为数学密集型计算而设计。CLA 可显著提升控制算法中常见的典型数学函数的性能。CLA 旨在与 C28x 中央处理器 并行执行实时控制算法，有效将计算性能加倍。这使得 CLA 非常适合以比 C28x 中央处理器 更高的周期性能改进来管理低级控制环路。CLA 还为快速中断响应而设计，并可直接访问诸如 ePWM、HRPWM、eCAP、eQEP、ADC、DAC、CMPSS、PGA、SDFM、SPI、LIN、FSI、PMBus、CLB 和 GPIO 等外设。

The CLA increases the overall compute bandwidth of the C2000 MCU and frees up the CPU for other tasks such as communications, systems, and diagnostics. Fig. 1.1 展示了 CLA 的主要优势：

- 降低的中断延迟
- 减少的控制环路延迟
3. 降低的 C28x CPU 带宽使用

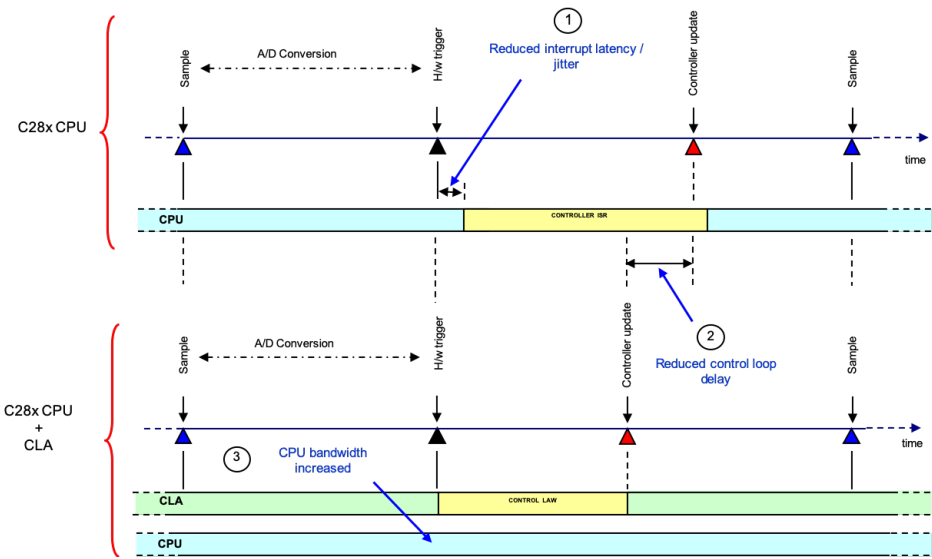


Fig. 1.1: Benefits of the CLA

Refer to these resources for more information on the CLA’s benefits and its role in an overall system:

- [Accelerators: Enhancing the Capabilities of the C2000™ MCU Family](#), section “Control Law Accelerator (CLA)”.
- [C2000™ Key Technology Guide](#)
- [CLA Usage in Valley Switching Boost Power Factor Correction \(PFC\) Reference Design](#)

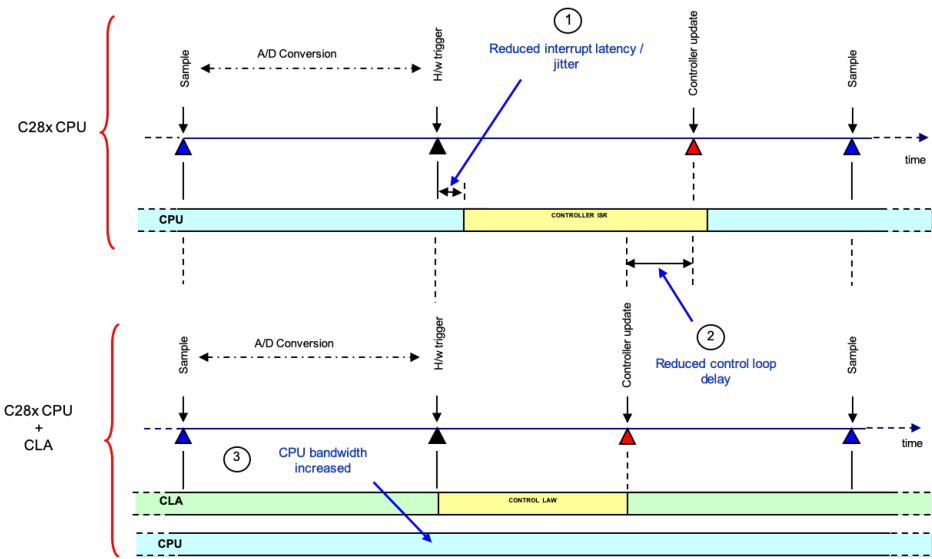


图 1.1: CLA 的好处

有关 CLA 的优势及其在整体系统中作用的更多信息，请参阅这些资源：

- [加速器：增强 C2000™ MCU 系列的能力](#)，段“控制律加速器 (CLA)”。
- [C2000™ 关键技术指南](#)
- [在谷切换升压功率因数校正 \(PFC\) 参考设计中对 CLA 的使用](#)

GETTING STARTED

This section is a roadmap for getting started using various resources such as workshops, examples and system use cases. The recommended sequence is:

- 1. Study the *workshops*.
- 2. Experiment with *examples*.
- 3. Review the *Frequently Asked Questions* and *Debugging Tips*.
- 4. Refer to the *System Use Cases* for application specific examples of leveraging the CLA.

2.1 Workshops

- If you are new to C2000 MCUs, start with the *C2000 Device Workshops*
- Review the training material in the *CLA Hands-On Workshop*
- Review Chapter 9, Control Law Accelerator in the *TMS320F28004x Microcontroller Workshop (pdf)*. Refer to *C2000™ F28004x Microcontroller Workshop* for the overall workshop.

Warning: New features may have been added to more recent devices or to the Compiler tools. Refer to the device-specific documents for the latest device (*Identifying Device-Specific CLA Features*) and compiler features (*Development Tools*).

Although the concepts taught in the workshops are applicable to all C2000 devices with the CLA, the projects may require older versions of the TI hardware and C2000 software. The workshop will list which hardware and software is supported.

入门

本节是使用各种资源（如研讨会、示例和系统用例）入门的路线图。推荐的顺序是：

- 1. Study the研讨会.
- 2. Experimentwith 示例.
- 3. Review the 常见问题解答and 调试提示.
- 4. 参考 *System Use Cases*，了解利用 CLA 的特定应用示例。

2.1 研讨会

- 如果您是 C2000 微控制器，建议从 *C2000 Device Workshops* 开始
- 在 *CLA Hands-On Workshop* 中查看培训材料
- 在 *TMS320F28004x Microcontroller Work-* 中查看第9章，Control Law Accelerator shop (pdf)。参考 *C2000™ F28004x Microcontroller Workshop* 了解整体研讨会。

警告： 可能在更新的器件或编译器工具中添加了新特性。有关最新的器件（*识别设备特定的 CLA 特性*）和编译器特性（*开发工具*）的详情，请参考特定器件文档。

虽然研讨会中教授的概念可适用于所有带有 CLA 的 C2000（C2000（德州仪器C2000系列））器件，但这些项目可能需要较旧版本的 TI 硬件和 C2000 软件。研讨会将列出哪些硬件和 软件 is 支持。

2.2 Development Tools

The CLA is supported by TI’s Code Composer Studio (**CCS**). Using CCS, both the C28x and CLA code can be developed and debugged within one integrated development environment.

2.2.1 Compiler and Assembly Tools

TI provides the CLA compiler as part of the C2000 Code Generation Tools. The CLA compiler and assembler are invoked automatically for CLA code. The Code Generation Tools are bundled in Code Composer Studio and can also be downloaded from: [C2000 code generation tools - compiler](#).

CLA firmware can be written using C code or assembly. The recommendation is to use C code in your firmware as much as possible. If further optimizations are necessary, then assembly code can also be used.

Features of the CLA tools can be found in the following User’s Guides:

- [TMS320C28x Optimizing C/C++ Compiler User’s Guide](#)
Describes the CLA compiler, its features and restrictions
- [TMS320C28x Assembly Language Tools User’s Guide](#)
Describes the extensions the assembler creates to support the CLA code. For example, command file sections and scratchpad sections.

2.2.2 MathWorks Embedded Coder Support

Visit the [MathWorks website](#) for information. MathWorks offers [examples specifically for the CLA](#)

2.3 Examples

1. Download the latest version of [C2000Ware](#). For those new to C2000ware, refer to the [C2000Ware](#) chapter in the [C2000™ Software Guide](#) and this training video: [Introduction to C2000Ware](#)
2. Select an example project
 - Basic math examples: included with the CLAMath Library in \libraries\math\CLAMath
 - Device-specific examples: in the driverlib or device_support directories
3. Review concepts presented in the Hands on Workshop while working with the project
 - Configuration and CLA initialization performed by the C28x

2.2 开发工具

The CLA is supported by TI’s Code Composer Studio (**CCS**). Using CCS, both the C28x and CLA code can be developed and debugged within one integrated development environment.

2.2.1 编译器与汇编工具

TI provides the CLA compiler as part of the C2000 Code Generation Tools. The CLA compiler and assembler are invoked automatically for CLA code. The Code Generation Tools are bundled in Code Composer Studio and can also be downloaded from: [C2000 code generation tools - compiler](#).

CLA 固件可以使用 C 代码或汇编编写。建议在固件中尽可能多地使用 C 代码。如果需要进一步优化，则也可以使用汇编代码。

CLA 工具的特性可在以下用户指南中找到：

- [TMS320C28x 优化 C/C++ 编译器用户指南](#)
描述 CLA 编译器、其特性和限制
- [TMS320C28x 汇编语言工具用户指南](#)
描述汇编器为支持 CLA 代码所创建的扩展。例如，命令文件段和临时存储区段。

2.2.2 MathWorks Embedded Coder 支持

访问 [MathWorks 网站](#) 以获取信息。MathWorks 提供专门针对 CLA 的示例

2.3 示例

1. 下载最新版本的[C2000Ware](#)。对于初次接触 C2000ware 的用户，请参阅[C2000Ware](#) 章节，位于 [C2000TMSoftwareGuide](#)，以及本培训视频：[Introductionto C2000Ware](#)
2. 选择一个示例工程
 - 基本数学示例：包含在 CLAMath 库 的 \libraries\math\CLAMath
 - 特定器件示例：位于 driverlib 或 device_support 目录中
3. 在使用该工程时，回顾 Hands on Workshop 中介绍的概念
 - 配置 和 CLA 初始化 由 C28x 执行

- Set and remove breakpoint(s) in CLA code
- Single step CLA code and observe changes in memory or registers
- Observe how data is shared between the C28x and CLA (see also *How is data shared between the CLA and C28x?*)

2.4 FAQs and Debugging Tips

Refer to the *Frequently Asked Questions* for answers to commonly asked questions.

The *Debugging Tips* section provides help for common problems encountered when developing and debugging CLA code. Before using the information in this section, it is strongly recommended that you review the information in the CLA Hands-On Workshop.

2.5 System Use Cases

- The [DigitalPower Software Development Kit \(SDK\)](#) lists TI reference designs that leverage the CLA in the Features section.
- Refer to [CLA Usage in Valley Switching Boost Power Factor Correction \(PFC\) Reference Design](#) for a training video describing the usage of CLA in [TIDM-1022](#).

- 在 CLA 代码 中 设置和移除 断点
- 单步执行 CLA 代码并观察内存或寄存器的变化
 - 观察数据在 C28x 与 CLA 之间如何共享（另见 *How is data shared between the CLA and C28x?*）

2.4 常见问题与调试技巧

查阅[常见问题解答](#) 以获取对常见问题的解答。

The *Debugging Tips* 部分提供在开发和调试 CLA 代码时遇到的常见问题的帮助。在使用本节中的信息之前，强烈建议您先查看 CLA 实操研讨会 中的信息。

2.5 系统用例

- 该[DigitalPower 软件开发工具包（SDK）](#) 在功能部分列出了利用 CLA 的 TI 参考设计。
- 请参阅 [在谷切换升压功率因数校正 \(PFC\) 参考设计中对 CLA 的使用](#) 以观看描述 CLA 在 [TIDM-1022](#) 中使用的培训视频。

IDENTIFYING DEVICE-SPECIFIC CLA FEATURES

This chapter provides pointers to determine which features are implemented in the CLA available on a given device.

3.1 CLA Types

Table 3.1 lists describes the types of the CLAs and maps types to devices. The type of the CLA in a particular C2000 MCU is also indicated in the device datasheet.

IDENTIFYING DEVICE-SPECIFIC CLA FEATURES

本章提供用于确定在指定设备 上可用的 CLA 中实现了哪些特性的指针。

3.1 CLA 类型

Table 3.1 列举了 CLA 的类型并将类型映射到器件。特定 C2000 MCU 中 CLA 的类型也在设备数据手册中予以标明。

Table 3.1: CLA Types

Type	Description	Devices
0	Original CLA module type	• F2803x (Only supports data RAM0 and 1 and does not allow CPU access when CLA dataRAM is enabled.) • F2805x, F2806x (Adds supports for data RAM2 and adds option to enable CPU access to data RAMs.)
1	• Increased Program address reachability to 16-bits • Added instructions to support the new address reach • Added two new offset addressing modes • CLA program memory is now user selectable and can reside anywhere in the lower 64k address space (excluding the M0 and M1 space). • Handing control to the CLA and assigning triggers to a task is now done at the system level. • A task can now fire an interrupt to main CPU mid execution.	F2807x, F2837xD, F2837xS
2	• Added background-code mode,that can run task like communications & clean-up routines in background • Background task runs continuously until disable or device/softreset • Background task can be triggered by a peripheral or software • Other foreground tasks can interrupt background task in the priority order defined • Added provision for making sections of background code uninterruptible • Added debug enhancements which has true software breakpoint support, where CLA re-fetches from the same address where halted during debug stop.	F28004x, F2838xD, F2838xS

表 3.1: CLA 类型

类型说明	设备
0 原始 CLA 模块类型	•F2803x (仅支持升级) 端口 数据 RAM0 and 1 and does not 允许 CPU 在 CLA 时 访问 数据RAM is en- 已启用。) • F2805x , F2806x (添加 支持 for data RAM2 并添加选项以启用中 央处理器访问 到数据 RAM。)
1 • 将程序地址可达范围增加到16位 • 已添加指令以支持新的 address reach • 添加了两种新的 offset addressing modes • CLA program memory 现在可由用户选择, 并且可以驻留在 下位64K地址空间的任何位置 (不包括 theM0 和 M1 空间)。 • 将控制权移交给 CLA 并将触发器分配给任务现在在系统级 别完成。 • 任务现在可以在执行中途向 main CPU 触发一个中断。	F2807x , F2837xD, F2837xS
2 • 添加了后台代码模式, 可在后台运行诸如通信 & 清理例 程之类的任务 • 后台任务持续运行, 直到被禁用或设备/softreset • 后台任务可以由外设或软件触发 • 其他前台任务可按定义的优先级顺序中断后台任务 • 为使后台代码的段变为不可中断添加了规定 • 添加了调试增强功能, 具有真正的软件断点支持, 其中 CLA 在 debug stop 期间从相同的地址重新取指 (re-fetches)	F28004x , F2838xD, F2838xS

3.2 Device-Specific Features

- **Device Technical Reference Manual (TRM)** Detailed CLA feature information for the specific device. This includes:
 - Configuration registers.
 - Pipeline.
 - Assembly instruction set.
- **Device Datasheet** Specifics related to the integration of the CLA into a device. This includes:
 - The device block diagram (bus connections).
 - Memories and peripherals accessible by the CLA (memory and register maps).
 - CLA frequency.

Note: To quickly find the device TRM and datasheet, browse to the product folder for your device. The product folder URL is of the form: **www.ti.com/product/<partnumber>**. Examples:

- www.ti.com/product/TMS320F28069
- www.ti.com/product/TMS320F28379D

The first document, at the top of the product folder, is the datasheet and the second is the TRM.

3.2 设备特定功能

- **Device Technical Reference Manual (TRM)** 特定器件的详细 CLA 特性信息。包括：
 - 配置寄存器。
 - 流水线。
 - 汇编指令集。
- **Device Datasheet** 与将 CLA 集成到器件相关的具体信息。此处包括：
 - 器件框图（总线连接）。
 - CLA 可访问的存储器和外设（内存和寄存器映射）。
 - CLA 频率。

Note: 要快速查找设备 TRM 和数据手册，请浏览到您器件的产品文件夹。产品文件夹网址格式为：**www.ti.com/product/<partnumber>**。示例：

- www.ti.com/product/TMS320F28069
- www.ti.com/product/TMS320F28379D

第一份，位于产品文件夹顶部，是数据手册，第二份是 TRM。

FREQUENTLY ASKED QUESTIONS

4.1 Is the CLA independent from the C28x CPU?

Yes. Once the CLA is configured by the main CPU it can execute algorithms independently. The CLA has its own bus structure, register set, pipeline and processing unit. In addition the CLA can access a host of peripheral registers directly. This makes it ideal for handling time-critical control loops and filtering or math algorithms.

4.2 Can the C28x and CLA be synchronized?

Synchronization is done by starting CLA execution either by a peripheral interrupt or by writing to a specific register. In addition the CLA can interrupt the main CPU.

4.3 Is the CLA programmable?

Yes. There are no algorithms built into the CLA hardware. The CLA is completely programmable in C or assembly.

4.4 Is there a C compiler for the CLA?

Yes. The TMS320C28x Code generation tools includes support for compiling CLA C code. Because of CLA architecture and programming environment constraints, the C language supported on the CLA has some restrictions. These are documented in the [TMS320C28x Optimizing C/C++ Compiler User’s Guide](#), Chapter “CLA Compiler”.

常见问题解答

4.1 CLA 是否独立于 C28x 中央处理器？

是的。一旦主CPU配置了 CLA，CLA 就可以独立执行算法。CLA 有其自己的总线结构、寄存器组、流水线和处理单元。此外，CLA 可以直接访问大量外设寄存器。这使其非常适合处理时间关键控制回路以及滤波或数学算法。

4.2 C28x 与 CLA 能否同步？

同步是通过由外设中断启动 CLA 执行或写入特定寄存器来完成的。此外，CLA 可以中断主 CPU。

4.3 CLA 是否可编程？

是的。CLA 硬件中没有内置的任何算法。CLA 完全可编程，可以使用 C 语言或汇编语言。

4.4 是否有用于 CLA 的 C 编译器？

是的。TMS320C28x 代码生成工具包含对编译 CLA 的 C 代码的支持。由于 CLA 架构和编程环境的限制，CLA 上支持的 C 语言存在一些限制。这些内容已记录在[TMS320C28x Optimizing C/C++Compiler User’s Guide](#)，“CLA Compiler”一章。

4.5 How is data shared between the CLA and C28x?

The examples in [C2000Ware](#) demonstrate data exchange between the C28x and the CLA. This is done through RAM blocks that both of the CPUs can directly access. Since the CLA and the C28x code reside within same project, sharing data can be accomplished using these steps:

1. Create a shared header file with common constants and variables. Include this file in both the C28x C and CLA code.
2. Use data section pragma statements and the linker file to place the variables in the appropriate RAM.
3. Define shared variables in your C28x .c code. C28x and CLA shared global variables must be defined in the C28x .c code, and not the CLA code. The way data pages work on each device does not match. It is more constrained on the C28x side. Thus, data defined on the C28x side can be accessed on the CLA side, but not the other way around.
4. Initialize variables in the CPU to CLA message RAM with the main CPU.
5. Initialize variables in the CLA to CPU message RAM with a CLA task. This initialization task can be started via the main C28x software.

4.6 How are data types different on C28x and CLA?

See also:

C28x and CLA data types are documented in the version specific Compiler User’s Guide. ([Com-
piler and Assembly Tools](#))

To avoid ambiguity when sharing data between CLA and C28x, it is strongly recommended that you use type declarations that include size information. For example, don’t use int. Use int32_t and uint16_t which are defined in stdint.h for both the C28x and CLA.

For example:

```
1 //
2 // in the shared header file and main .c file
3 // Does not indicate the size of the variable
4 // An unsigned int on C28x is 16-bit and an unsigned int on CLA is 32-
  ↪bits
5 //
6 unsigned int t0;
7 unsigned int t1;
8 //
9 // Instead use C99 data types with size information
10 //
11
```

4.5 CLA 与 C28x 之间如何共享数据？

[C2000Ware](#) 中的示例演示了 C28x 与 CLA 之间的数据交换。这是通过两个中央处理器（CPU）都可以直接访问的 RAM 块来完成的。因为 CLA 和 C28x 代码位于同一项目中，共享数据可以通过以下步骤来实现：

1. 创建一个包含常用常量和变量的共享头文件。将此文件包含在两个文件中 C28x C代码 和 CLA 代码 中。
2. 使用 data section pragma 语句和 链接器文件 将变量放置在适当的 RAM.
3. 在你的 C28x .c 代码中定义共享变量。C28x 和 CLA 共享全局变量必须在 C28x .c 代码中定义，而不是在 CLA 代码中。每个器件上数据页的工作方式不一致。在 C28x 端受限更多。因此，在 C28x 端定义的数据可以在 CLA 端访问，但反之则不行。
4. 在主CPU中将变量初始化到CPU到CLA消息RAM。
5. 使用CLA任务将变量初始化到CLA到CPU消息RAM。此初始化任务可以通过主C28x软件启动。

4.6 C28x和CLA上的数据类型有何不同？

另见：

C28x 和 CLA 数据类型已记录在特定版本的 编译器用户指南 中。 ([Com-
piler and AssemblyTools](#))

为了在 CLA 与 C28x 之间共享数据时避免歧义，强烈建议使用包含大小信息的类型声明。例如，不要使用 int。请使用在 stdint.h 中为 C28x 和 CLA 定义的 int32_t 和 uint16_t。 — —

例如：

```
1 //
2 // in the shared header file and main .c file
3 // Does not indicate the size of the variable
4 // An unsigned int on C28x is 16-bit and an unsigned int on CLA is 32-
  ↪bits
5 //
6 unsigned int t0;
7 unsigned int t1;
8 //
9 // Instead use C99 data types with size information
10 //
11
```

```
12 #include <stdint.h>
13 uint16_t t0;
14 uint16_t t1;
```

- Integers:**
- For CLA an int is 32-bits
 - For C28x an int is 16-bit

Pointers:

Pointers are interpreted differently on the C28x and the CLA. The C28x treats them as 32-bit data types (address bus size being 22-bits wide can only fit into a 32-bit data type) while the CLA only has an address bus size of 16 bits.

Assume the following structure is declared in a shared header file(i.e. common to the C28 and CLA) and defined and allocated to a memory section in a .c file:

Listing 4.1: Pointer Size Problem

```

/
↳ *****
    shared.h
    C28x and CLA Shared Header File
↳ *****/

typedef struct{
    float a;
    float *b;
    float *c;
}foo;

/
↳ *****
    main.c
    Main C28x C-code Source File
↳ *****/

#pragma (X, "CpuToCla1MsgRam")           // Assign X to section
↳CpuToCla1MsgRam
foo X;

/
↳ *****
    test.cla
    CLA C-code Source File
```

```
12 #include <stdint.h>
13 uint16_t t0;
14 uint16_t t1;
```

- 整数:**
- 对于 CLA, int 是 32 位
 - 对于 C28x, int 是 16 位

指针:

指针在 C28x 和 CLA 上的解释不同。C28x 将它们视为 32 位数据类型（地址总线宽度为 22 位 —— 只能适配到 32 位数据类型），而 CLA 的地址总线宽度只有 16 位。

假设以下结构在一个共享头文件中声明（即对 C28 和 CLA 通用），并在一个 .c 文件中定义并分配到一个内存段：

清单 4.1: 指针大小问题

```

/
↳ *****
    shared.h
    C28x and CLA Shared Header File
↳ *****/

typedef struct{
    float a;
    float *b;
    float *c;
}foo;

/
↳ *****
    main.c
    Main C28x C-code Source File
↳ *****/

#pragma (X, "CpuToCla1MsgRam")           // Assign X to section
↳CpuToCla1MsgRam
foo X;

/
↳ *****
    test.cla
    CLA C-code Source File
```



```

*****/
__interrupt void Cla1Task1 ( void )
{
    float f1,f2;
    f1 = *(X.b);
    f2 = *(X.c);    //Pointer incorrectly dereferenced
                   //Tries to access location 0x1503 instead
                   //of 0x1504
}
```

Assume that the C28x compiler will allocate space for X at the top of the section CpuTo-
Cla1MsgRam as follows:

Element	Description	C28x Address
X.a	a is a 32-bit float in address	0x1500-0x1501
X.b	b is a 32-bit pointer in address	0x1502-0x1503
X.c	b is a 32-bit pointer in address	0x1504-0x1505

The CLA will interpret this structure differently. The CLA treats pointers “‘b’” and “‘c’” as
16-bits wide and, therefore, incorrectly dereferences pointer c.

Element	Description	CLA Address
X.a	a is a 32-bit float in address	0x1500-0x1501
X.b	b is a 16-bit pointer in address	0x1502
X.c	c is a 16-bit pointer in address	0x1503

The solution to this is to declare a new pointer as follows:

Create a new pointer “‘CLA_FPTR’” which is a union of a 32-bit integer and a pointer to a float.
The CLA compiler recognizes the size of the larger of the two elements (the 32 bit integer) and
therefore aligns the pointer to the lower 16-bits. Now both the pointers “‘b’” and “‘c’” will
occupy 32-bit memory spaces and any instruction that tries to de-reference pointer c will access
the correct address 0x1504.

Listing 4.2: Pointer Size Solution

```

*****
shared.h
C28x and CLA Shared Header File
*****/
typedef union{
    float *ptr; //Aligned to lower 16-bits
    Uint32 pad; //32-bits
}CLA_FPTR;
```

```

*****/
__interrupt void Cla1Task1 ( void )
{
    float f1,f2;
    f1 = *(X.b);
    f2 = *(X.c);    //Pointer incorrectly dereferenced
                   //Tries to access location 0x1503 instead
                   //of 0x1504
}
```

假设 C28x 编译器将为 X 在段 CpuTo- Cla1MsgRamas 的顶部分配空间，如下：

元素	说明	C28x 地址
X.a	a 是 地址中的 32 位浮点数	0x1500-0x1501
X.b	b 是 地址 中的 32 位指针	0x1502-0x1503
X.c	b 是 地址 中的 32 位指针	0x1504-0x1505

CLA 将以不同的方式解释此 结构。CLA 将指针 “ ‘b’ ” 和 “ ‘c’ ” 视为
16位 宽，因此错误地解引用了指针 c。

元素	说明	CLA 地址
X.a	a 是位于 地址 的 32 位浮点数	0x1500-0x1501
X.b	b 是一个 16-bit 指针，位于 地址	0x1502
X.c	c 是一个 16-bit 指针，位于 地址	0x1503

对此的解决方法是按如下方式声明一个新的指针：

创建一个新的指针 “ ‘CLA_FPTR’ ”，它是一个 32 位整数和一个指向浮点数的指
针的联合体。CLA 编译器识别出两个元素中较大的大小（32 位整数），因此将指针对齐
到低 16 位。现在指针 “ ‘b’ ” 和 “ ‘c’ ” 都将占据 32 位内存空间，任何
尝试解引用指针 c 的指令都将访问正确的地址 0x1504。

清单 4.2：指针大小解决方案

```

*****
shared.h
C28x and CLA Shared Header File
*****/
typedef union{
    float *ptr; //Aligned to lower 16-bits
    Uint32 pad; //32-bits
}CLA_FPTR;
```



```
typedef struct{
    float a;
    CLA_FPTR b;
    CLA_FPTR c;
}foo;

/*****
main.c
Main C28x C-code Source File
*****/
#pragma (X, "CpuToCla1MsgRam") //Assign X to section CpuToCla1MsgRam
foo X;

/*****
test.cla
CLA C-code Source File
*****/
__interrupt void Cla1Task1 ( void )
{
    float f1,f2;
    f1 = *(X.b.ptr);
    f2 = *(X.c.ptr); //Correct Access
}
```

4.7 What can trigger a task (also referred to as an ISR)?

The following sources can start (trigger) a task:

- 1. A peripheral interrupt
- 2. Software trigger
- 3. Another CLA task
- 4. On some devices a specific task can be designated as a background task. Refer to *Identifying Device-Specific CLA Features* for details.

The C28x configures the CLA. Part of the configuration is specifying what silicon resource trigger which tasks. There are generally two registers used to configure the triggers for a task.

- Older devices use the the MPISRCSEL1 register (i.e. 2803x, 2805x, 2806x).
- More recent devices use a system register CLA1TASKSRCSELx (2837xD/S, F28004x...).

```
typedef struct{
    float a;
    CLA_FPTR b;
    CLA_FPTR c;
}foo;

/*****
main.c
Main C28x C-code Source File
*****/
#pragma (X, "CpuToCla1MsgRam") //Assign X to section CpuToCla1MsgRam
foo X;

/*****
test.cla
CLA C-code Source File
*****/
__interrupt void Cla1Task1 ( void )
{
    float f1,f2;
    f1 = *(X.b.ptr);
    f2 = *(X.c.ptr); //Correct Access
}
```

4.7 什么可以触发任务（也称为 ISR）？

以下源可以启动（触发）一个任务：

- 1. 外设中断
- 2. 软件触发
- 3. 另一个 CLA 任务
- 4. 在某些器件上，特定任务可以指定为后台任务。参见识别设备特定的 *CLA* 特性了解详细信息。

C28x 配置 CLA。配置的一部分是指定哪个硅资源触发哪些任务。通常有两个寄存器用于配置任务的触发器。

- 较旧的设备使用 MPISRCSEL1 寄存器（即 2803x、2805x 系列、2806x）。
- 更新的器件使用一个系统寄存器 CLA1TASKSRCSELx (2837xD/S, F28004x...)。

See also:

- Examples in [C2000Ware](#).
- The device-specific Technical Reference Manual (TRM) documents which peripherals can trigger which CLA tasks. (*Identifying Device-Specific CLA Features*).

The trigger can be an interrupt or it can be the C28x CPU through software. It is important to understand the trigger source is only the mechanism by which the task is started. The trigger source does not limit what the task can do or what registers it accesses.

- The main C28x CPU
 - Can start a task by using the IACK #16bit instruction. For example IACK 0x0003 would flag interrupt 1 and interrupt 2. This is the same as setting bits in the force register (MIFRC)
- Another CLA task:
 - On some devices, the CLA does not have access to directly force another task to be flagged. There are a couple of options:
 - Interrupt the C28x and force the task as part of the interrupt service routine.
 - Write to the ePWM register to force an interrupt which in-turn forces a task to be flagged.

4.8 Is there support for nesting tasks?

The CLA has its own fetch mechanism and can run and execute a task independently of the CPU. Only one task is serviced at a time - there is no nesting of tasks unless the background task is enabled. On devices with CLA Type 2 supporting background tasks, one level of nesting is possible. See [CLA Types](#).

4.9 Is there a limitation on the code size for tasks?

Think of a task like an interrupt. It is a algorithm that is executed when the CLA receives an interrupt. The size of a task is only limited by the amount of program memory available and the CLA’s program counter.

The start address of a task is configurable. Each task has an associated interrupt vector (MVECT1 to MVECT8). For type 0 CLAs, ([CLA Types](#)) this vector holds the starting address (as an offset from the first program location) of the task. For all the other types, this vector holds the entire 16-bit address of the task.

- CLA Type 0:

另请参阅:

- 示例见 [C2000Ware](#).
- 器件特定的技术参考手册（TRM）记录了哪些外设可以触发哪些 CLA 任务。（识别设备特定的 [CLA 特性](#)）。

触发可以是一个中断，也可以是通过软件由 C28x 中央处理器 发起。重要的是要理解触发源只是启动任务的机制。触发源并不限制任务可以执行的操作或它访问的寄存器。

- 主 C28x 中央处理器
 - 可以使用 IACK 16 位指令来启动一个任务。例如 IACK 0x0003 会置位中断 1 和中断 2。这与在强制寄存器 (MIFRC) 中设置位是相同的
- 另一个 CLA 任务:
 - 在某些器件上，CLA 无法直接访问以强制另一个任务被置位。有几种选项:
 - 对 C28x 触发中断，并在中断服务例程中强制该任务。
 - 写入 ePWM 寄存器以强制一个中断，而该中断又会强制一个任务被置位。

4.8 是否支持任务嵌套？

CLA 有其自身的取指机制，可以独立于中央处理器（CPU）运行并执行任务。一次只服务一个任务——除非启用了后台任务，否则不支持任务嵌套。在支持后台任务的 CLA 类型 2 的器件上，允许一层嵌套。参见[CLA Types](#)。

4.9 任务的代码大小是否有限制？

将任务视为一个中断。当 CLA 接收到中断时，会执行该算法。任务的大小仅受可用程序存储器和 CLA 的程序计数器的限制。

任务的起始地址是可配置的。每个任务都有一个关联的中断向量（MVECT1 到 MVECT8）。对于类型 0 CLA，（[CLA Types](#)）该向量保存任务的起始地址（作为从第一个程序位置的偏移）。对于所有其他类型，该向量保存任务的整个 16位地址。

- CLA 类型 0:

The program space is limited to 12-bits or 4096 words. All CLA instructions are 32-bits, so within a 4k x 16 program space you can have ~2k CLA instructions.

- CLA Type 1 and above:

The program space is 16-bits wide, leaving the lower 64K words of space available as program space. Please refer to your device specific documentation for information on how to configure/allocate the memory spaces to the CLA.

The MSTOP instruction indicates the end of the task. After a task begins, the CLA will execute instructions until it encounters an “MSTOP” instruction.

4.10 Which peripheral registers can the CLA access directly?

Warning: Refer to the device-specific datasheet. If there is more than one CLA on a device, they may not all have the ability to connect to all of the same peripherals. (*Device-Specific Features*)

The answer is C2000 MCU family dependent. In general, more recently released device families support CLA access to more peripherals. The list below indicates where to look in the datasheet:

1. The device block diagram: indicates which peripherals are connected to the CLA bus.
2. The device level memory map: which memory blocks can be configured for CLA access
3. The external interface (EMIF) memory map: indicates any memory regions the CLA can access
4. The peripheral register memory maps: which specific peripheral registers the CLA has access to

Some examples:

- 2803x: Direct access to the ADC result, ePWM+HRPWM, and comparator registers.
- 2806x: Direct access to the ADC result, ePWM+HRPWM, eCAP, eQEP and comparator registers.
- 2807x: Direct access to the the ADC module(s) (including results), ePWM+HRPWM, eCAP, eQEP, comparator subsytem, DAC subsystem, SPI, McBSP, uPP, EMIF, GPIO
- 2837x: Direct access to the the ADC module(s) (including results), ePWM+HRPWM, eCAP, eQEP, comparator subsytem, DAC subsystem, SPI, McBSP, uPP, EMIF(s), GPIO

程序空间限制为 12 位 或 4096 字。所有 CLA 指令都是 32 位，因此在 4k x 16 程序空间中你可以拥有 ~2k CLA 指令。

- CLA 类型 1 及以上：

程序空间是 16位 宽，保留下部 64K 字 作为可用的程序空间。请参阅您的器件 特定文档， 了解如何 将内存 空间 配置 / 分配 给 CLA。

MSTOP 指令 表示 任务 结束。任务开始 之后，CLA 将执行 指令，直到遇到 “MSTOP ” 指令。

4.10 CLA 可以直接访问 哪些 外设 寄存器？

Warning: Refer to the device-specific datasheet. If there is more than one CLA on a device, they may not all have the ability to connect to all of the same peripherals. (*设备特定功能*)

答案取决于 C2000 MCU 系列。一般来说，最近发布的器件系列支持 CLA 访问更多外设。下面的列表指出了在数据手册中应该查看的位置：

1. 设备框图：指示哪些外设连接到 CLA 总线。
2. 器件级内存映射：哪些存储块可以配置为供 CLA 访问
3. 外部接口 (EMIF) 内存映射：指示 CLA 可以访问的任何内存区域
4. 外设寄存器内存映射：CLA 有权访问的具体外设寄存器的哪些部-
cess to

一些 示例：

- 2803x: 直接寻址 模数转换器（ADC） 结果、增强型脉宽调制（ePWM）+高分辨率脉宽调制（HRPWM）和比较器寄存器。

- 2806x: 直接寻址 模数转换器（ADC） 结果、增强型脉宽调制（ePWM）+高分辨率脉宽调制（HRPWM）、事件捕获模块（eCAP）、增强型正交编码器脉冲（eQEP）和比较器

寄存器。

- 2807x: 直接寻址 ADC 模块（包括结果）、增强型脉宽调制（ePWM）+高分辨率脉宽调制（HRPWM）、事件捕获模块（eCAP）、增强型正交编码器脉冲（eQEP）、比较器子系统、DAC 子系统、串行外设接口（SPI）、多通道缓冲串行端口（McBSP）、并行端口（uPP）、外部存储器接口（EMIF）、通用输入输出（GPIO）
- 2837x: 直接寻址 ADC 模块（包括结果）、增强型脉宽调制（ePWM）+高分辨率脉宽调制（HRPWM）、事件捕获模块（eCAP）、增强型正交编码器脉冲（eQEP）、比较器子系统、数模转换器（DAC）子系统、串行外设接口（SPI）、多通道缓冲串行端口（McBSP）、并行端口（uPP）、外部存储器接口（EMIF(s)）、通用输入输出（GPIO）

4.11 Can the CLA send an interrupt to the C28x?

Todo: check if there are new mechanisms on F28004x

Yes. The CLA can send an interrupt to the C28x as follows:

- End of Task:
The CLA will send an interrupt to the PIE (peripheral interrupt expansion block) to let the main CPU (C28x) know a task has completed. Each task has an associated vector in the PIE. This interrupt is automatically fired when the associated task completes. For example when task 1 completes, CLA1_INT1 in the PIE will be flagged. If this interrupt is not enabled, then the C28x will ignore it.
- Forced Interrupt:
This applies to CLA type 1 and later. (Note: when this option is enabled it automatically disables the end of task interrupt). The CLA has a Software Interrupt Capability, where a task can enable and force an interrupt to the main CPU. For example Task 1 can enable a software interrupt for task 2, by writing to the TASK2 bit of the CLA1SOFTINTEN register, and then forcing that interrupt by writing to the TASK2 bit of the CLA1SOFTINTFRC register.
- Overflow and Underflow:
There are dedicated interrupts in the C28x PIE for CLA floating-point overflow and underflow conditions.

4.12 Does the CLA have access to all memory blocks?

No, the memory blocks that are available for use by the CLA is device family dependent.

See also:

The memory map in the device-specific datasheet indicates which blocks the CLA can access. (*Device-Specific Features*)

- The main CPU can allocate specific memory blocks to the CLA
- The memory map indicates which memory blocks can be configured for CLA usage.
- In some devices the memory usage (program or data) is fixed. In other devices some memory blocks accessible by the CLA can be configured as program memory or data memory.

In addition, there are dedicated message RAMs with a fixed configuration. Look for these in the memory map as well. Two examples are given below. Some devices have additional message RAMs.

4.11 CLA 能向 C28x 发送中断吗?

Todo: 检查 F28004x 系列 上是否有新的机制

是的。CLA 可以按以下方式向 C28x 发送中断:

- 任务结束:
CLA 会向 PIE (外设中断扩展块) 发送中断, 以让主CPU (C28x) 知道某个任务已完成。每个任务在 PIE 中都有一个相关联的向量。当关联任务完成时, 该中断会自动触发。例如, 当任务 1 完成时, PIE 中的 CLA1_INT1 将被置位。如果该中断未被使能, 则 C28x 会忽略它。
- 强制中断:
This applies to CLA 类型 1 及以后。(注意: 当此选项被使能时, 它会自动禁用任务结束中断)。CLA 具有软件中断功能, 任务可以使能并触发发送到主CPU 的中断。例如, 任务 1 可以通过写入 CLA1SOFTINTEN 寄存器的 TASK2 位来使能任务 2 的软件中断, 然后通过写入 CLA1SOFTINTFRC 寄存器的 TASK2 位来触发该中断。
- 溢出和下溢:
在 C28x 的 PIE 中有专门的中断用于 CLA 浮点溢出和下溢情况。

4.12 CLA 是否可以访问所有存储块?

不, CLA 可用的存储块取决于器件系列。

另见:

器件特定数据手册中的存储映射指示 CLA 可以访问哪些块。 (*Device-Specific Features*)

- 主CPU可以将特定的存储块分配给CLA
- 存储映射指示哪些存储块可以配置为 CLA 使用。
- 在某些器件中, 存储使用 (程序或数据) 是固定的。在其他器件中, CLA 可访问的一些存储块可以配置为程序存储器或数据存储器。

此外, 还有具有固定配置的专用消息RAM。也可以在存储映射中查找这些。下面给出了两个示例。某些器件具有额外的消息RAM。

- CLA to CPU Message RAM: CLA can read/write, main CPU can only read
- CPU to CLA Message RAM: CPU can read/write, CLA can only read

4.13 If C28x and CLA can access the same device resource, which has priority?

The device automatically arbitrates the access to the resource. That is, there is a defined order of priority.

See also:

Search for the word “arbitration” in the device-specific Technical Reference Manual (TRM) for more information. (*Device-Specific Features*).

Keep in mind if the main CPU performs a read-modify-write operation on a peripheral register and between the read and the write the CLA modifies the same register, the changes made by the CLA can be lost. In general it is best to not have both processors writing to the same registers.

If the CLA is configured to respond to a peripheral interrupt, then the interrupt will still go to the C28x PIE. This enables both CPUs to respond if needed.

4.14 How can I measure the duration of a task?

- Option 1: Use a timer:

One method is to use a free running timer to capture the number of cycles the task execution took. The CLA has access to the PWM timers. As an example you could capture the number of cycles in C as follows:

1. At the start of the task, CLA reads the timer
2. Execute the task
3. At the end of the task, CLA reads the timer again
4. Calculate the number of cycles between (1) and (3)

```
ct1 = EPwm1Regs.TBCTR;
//
//..... code to be measured here .....
//
ct2 = EPwm1Regs.TBCTR;
delta_ct = ct2 - ct1;
```

- Option 2: Use a GPIO pin:

- CLA到CPU消息RAM：CLA 可读/写，主CPU 只能读取
- CPU到CLA消息RAM：CPU可读/写，CLA仅可读

第 4.13 节 如果 C28x 与 CLA 可以访问同一器件资源，哪个具有优先级？

该器件会自动对对资源的访问进行仲裁。也就是说，存在定义好的优先级顺序。

另请参见：

Search for the word “arbitration” in the device-specific Technical Reference Manual (TRM) for more information. (*Device-SpecificFeatures*).

请记住，如果主CPU对外设寄存器执行读-改-写 操作，并且在读和写之间CLA修改了相同的寄存器，则CLA所做的更改可能会丢失。一般来说，最好不要让两个处理器都写入相同的寄存器。

如果CLA被配置为响应外设中断，则中断仍然会发送到C28xPIE。这使得两个中央处理器（CPU）在需要时都可以响应。

4.14 我如何测量任务的持续时间？

- 选项 1：使用定时器：

一种方法是使用自由运行定时器来捕获任务执行所花费的周期数。CLA可以访问PWM 定时器。例如，您可以按如下方式在 C 中捕获周期数：

1. 在任务开始时，CLA 读取定时器
2. 执行任务
3. 在任务结束时，CLA 再次读取定时器
4. 计算（1）和（3）之间的周期数

```
ct1 = EPwm1Regs.TBCTR;
//
//..... code to be measured here .....
//
ct2 = EPwm1Regs.TBCTR;
delta_ct = ct2 - ct1;
```

- 选项 2：使用通用输入输出（GPIO）引脚：

On some devices, the CLA has direct access to GPIO pins. In this case, instead of reading a timer, the CLA could toggle a pin before and after the code to be measured. Monitor the pin using an oscilloscope to measure the delta in time.

- Accounting for Overhead

There will be some overhead due to triggering the CLA task and notifying the C28x that it has completed. In order to test this overhead, you could use a timer on the C28x to time the start and stop of the CLA task.

1. Read the timer using the C28x
2. Start the CLA task via the C28x Software
3. Wait for the MIRUN bit to clear
4. Read the timer using the C28x
5. Calculate the delta between (1) and (4)
6. Also refer to this post: <https://e2e.ti.com/support/microcontrollers/c2000/f/171/t/738758>

4.15 Can the C28x terminate a CLA task?

Yes. If a interrupt has been flagged, but the task has not yet run, the main CPU can clear the flag using the MICLR register.

If the task is already running then a soft reset (in MCTL) will terminate the task and clear the MIER register. If you want to clear all of the CLA registers you can use the hard reset option in the MCTL register.

4.16 What does the “Symbol, X, referenced in a.obj, assumes that data is blocked but is accessing non-blocked data in b.obj. Runtime failures may result” warning from the Linker mean?

This is a diagnostic added in C2000 code generation tools v20.2.x.LTS to detect when data accesses that assume data is blocked are made to non-blocked definitions.

The common case for this is a global CLA definition that is used in C28 code. For more background refer to [20.2.x LTS README entry for the new diagnostic](#).

在某些设备上, CLA 对通用输入输出 (GPIO) 引脚有直接寻址。在这种情况下, CLA 可以在要被测量的代码之前和之后切换一个引脚, 而不是读取定时器。使用示波器监测该引脚以测量时间差。

开销考虑

触发 CLA 任务 并 通知 C28x 已完成 会产生一些 开销。为了测试此 开销, 您可以在 C28x 上使用一个 定时器 来计时 CLA 任务 的开始和结束。

1. 使用 C28x 读取定时器
- 通过 C28x 软件 启动 CLA 任务
3. 等待 MIRUN 位 清除
4. 使用 C28x 读取定时器
5. 计算 (1) 和 (4) 之间的差值
6. Also refer to this post: <https://e2e.ti.com/support/microcontrollers/c2000/f/171/t/738758>

4.15 C28x 能终止 CLA 任务吗?

是的。如果一个 interrupt 已经被置位, 但任务尚未运行, 主CPU 可以使用 MICLR 寄存器清除该 标志。

如果任务已经在运行, 则在 MCTL 中进行的 soft reset 将终止任务并 清除 MIER 寄存器。如果您想 清除所有 CLA 寄存器, 可以在 MCTL 寄存器 中使用 硬复位选项。

4.16 “在 a.obj 中被引用的符号 X 假定数据被屏蔽但正在访问 b.obj 中未被屏蔽的数据。可能导致运行时故障” 的链接器警告是什么意思?

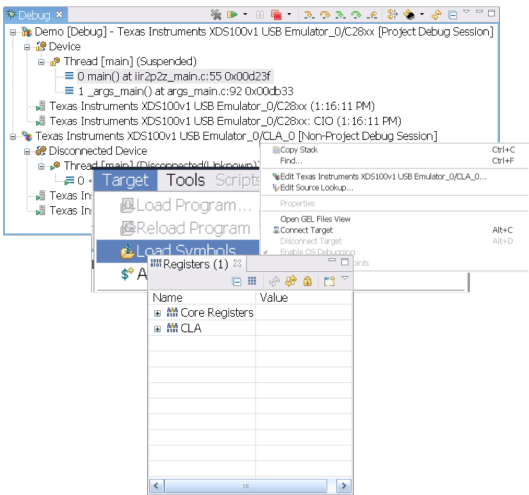
这是在 C2000 代码生成工具 v20.2.x.LTS 中添加的诊断, 用于检测假定数据被屏蔽的访问是否针对了未被屏蔽的定义。

对于这种情况的常见示例是用于 C28 代码的全局 CLA 定义。有关更多后台信息, 请参阅 [20.2.x LTS README 条目 \(用于新诊断\)](#)。

DEBUGGING TIPS

The CLA can halt, single-step and run independently from the main C28x CPU. Both the CLA and the C28x are debugged from the same JTAG port.

- When a debug session is launched, the debug view (window within CCS) will have entries for C28x and CLA.
- Clicking on CLA changes the context of all windows in CCS to reflect CLA data.
- Connecting to the CLA core from the debug perspective enables single stepping
- Load Program Symbols onto the CLA from Target menu options to begin debugging



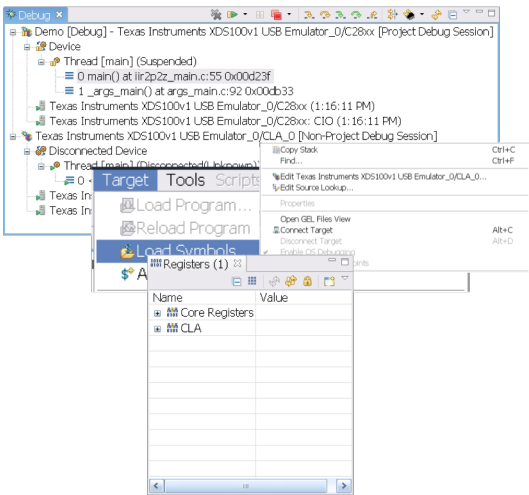
Debugging CLA code

1. Insert a breakpoint into CLA code
 - An __mdebugstop() intrinsic is a CLA breakpoint. It will place the instruction MDE-BUGSTOP at that position in assembly.
 - If not connected to the CLA core (i.e. single step disabled), the intrinsic behaves as a MNOP (no operation)

调试提示

CLA 可以独立于主 C28x 中央处理器暂停、单步执行和运行。CLA 和 C28x 均通过相同的 JTAG 接口进行调试。

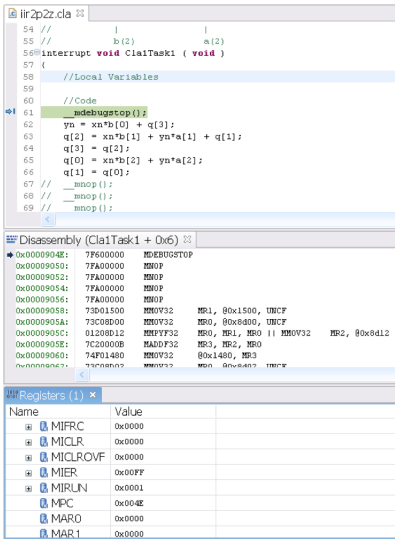
- 当启动调试会话时，调试视图（CCS 窗口内）将会列出 C28x 与（AND）CLA。
 - 单击 CLA 会将 CCS 中所有窗口的上下文更改为反映 CLA 数据。
 - 从调试透视连接到 CLA 内核可启用单步执行
- 从 目标（Target）菜单 选项将 加载程序符号 到 CLA 上以开始调试



调试 CLA 代码

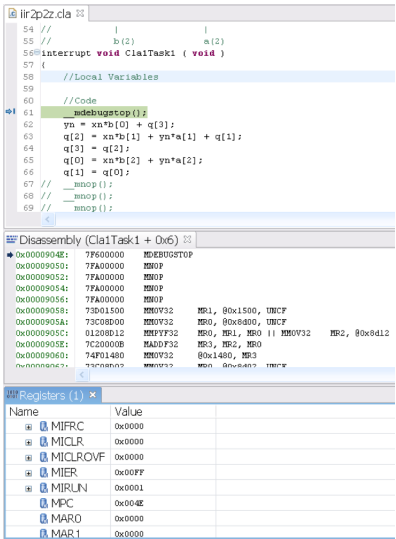
1. 在 CLA 代码中插入断点
 - An__mdebugstop() 内建函数是一个 CLA 断点。它将在汇编语言的该位置放置 MDEBUGSTOP 指令。
 - 如果未连接到 CLA 内核（即单步执行被禁用），该内建函数的行为等同于 MNOP（无操作）

- 2. Start the task: CLA will execute code until the MDEBUSTOP is in D2
- 3. Single step or run to the next breakpoint: Single stepping moves the CLA pipeline one cycle at a time



Note: The debugger cannot perform ‘run to line’ or ‘step over’ when debugging CLA code.

- 2. 启动任务：CLA 将执行代码直到 MDEBUSTOP 指令 位于 D2
- 3. 单步执行或运行到下一个断点：单步执行使 CLA 流水线每次移动一个周期



Note: 调试器在调试 CLA 代码时无法执行“运行到行”或“跳过”操作。

5.1 Help! Why doesn’t my CLA code work?

Place a breakpoint in the CLA code. Make sure the CLA is connected and see if you hit the breakpoint. Step through the code to try and narrow down the problem.

If the breakpoint is never hit, then the task may not be starting. Check the CLA set-up (vectors, tasks are enabled, etc..)

5.2 My CLA task never starts.

Try starting the task with software to narrow down the issue. This will help confirm the task vector is setup correctly.

If the task fails to start for a peripheral trigger, check when the peripheral initialization is done. A CLA task only triggers on a level transition (an edge) of the configured interrupt source. If the peripheral triggers an interrupt before the CLA is initialized, then it can be missed. One way to get around this is to clear the interrupt flags from the peripherals.

5.1 求助！为什么我的 CLA 代码不起作用？

在 CLA 代码中设置一个断点。确保 CLA 已连接，看看是否命中断点。逐步执行代码以尝试缩小问题范围。

如果从未命中断点，则任务可能未启动。检查 CLA 配置（向量、任务已启用等..）

5.2 我的 CLA 任务 从未启动。

尝试使用 software 启动该 task 以缩小问题范围。这将有助于确认 task vector 已正确设置。

如果任务因 peripheral trigger 未能启动，请检查外设初始化何时完成。CLA 任务仅在配置的中断源的电平变换（边沿）时触发。如果外设 在 CLA 已初始化之前触发中断，则该中断可能会被错过。解决此问题的一种方法是清除外设的中断标志。

5.3 Why can't I force a task using software (IACK)?

Verify the following:

- 1. This feature is enabled in the MCTL register. This register is EALLOW protected.
- 2. The interrupt is enabled in the MIER register. This register is EALLOW protected.
- 3. Software is configured as the task trigger. Refer to the device-specific TRM.
- 4. You are using the correct argument to IACK. For example IACK #0x0003 would flag interrupt 1 (bit 0) and interrupt 2 (bit 1).
- 5. Refer to the SW examples in [C2000Ware](#).

5.4 Why does my Code Composer Studio (CCS) GEL file enable the CLA clock on reset?

When you reset the device, the CLA breakpoints are disabled and the CLA clock itself is disabled. To make debugging CLA code easier, CCS will watch for a reset condition and re-enable CLA breakpoints after a reset. For this to occur the CLA clock itself must be re-enabled. If you do not like this behavior, the line can be commented out or removed from the GEL file.

5.5 Can I start a task from the debug window?

You can use the CLA force register MIFRC to start tasks. This register is available in the CLA register window.

5.6 After a “run”, why did the CLA begin executing another task instead of stopping at MSTOP?

If there is a task both pending and enabled when you run to the MSTOP, the pending task will automatically begin execution. To keep this from happening you can modify the MIER register so that no tasks are enabled.

5.3 为什么我无法使用软件（IACK）强制启动任务？

验证以下内容：

- 1. 该功能在 MCTL 寄存器 中使能。该寄存器受 EALLOW（寄存器写保护机制）保护。
- 2. 中断在 MIER 寄存器中被使能。该寄存器受 EALLOW（寄存器写保护机制）保护。
- 3. 软件被配置为任务触发器。参考器件特定的 TRM（技术参考手册）。
- 4. 你向 IACK 使用了正确的参数。例如 IACK #0x0003 会将中断 1（位 0）和中断 2（位 1）标记为标志。
- 5. 参见 [C2000Ware](#) 中的软件示例。

5.4 为什么我的 Code Composer Studio (CCS) GEL 文件在复位时使能 CLA 时钟？

当您复位器件时，CLA 断点和 CLA 时钟本身都会被禁用。为了便于调试 CLA 代码，CCS 会监视复位条件并在复位后重新使能 CLA 断点。要实现此行为，必须重新使能 CLA 时钟本身。如果您不喜欢此行为，可以在 GEL 文件中将该行注释掉或删除。

5.5 我可以从调试窗口启动任务吗？

您可以使用 CLA 强制寄存器 MIFRC 来启动任务。该寄存器可在 CLA 寄存器窗口中获得。

5.6 在“运行”之后，为什么 CLA 开始执行另一个任务而不是在 MSTOP 停止？

如果在运行到 MSTOP 时同时存在挂起且已启用的任务，挂起的任务将会自动开始执行。为避免发生这种情况，您可以修改 MIER 寄存器，使没有任务被使能。

5.7 Why are the variables in the CLA code not updating as expected?

A reason is that the `.scratchpad` and `.bss_cla` sections are incorrectly allocated to read only CLA Program memory. The task will run, but variables allocated to the `.bss_cla` section will not be updated. Moreover, the `.scratchpad` section is used by the compiler and allocating to read only memory can result in undefined behavior.

5.7 为什么 CLA 代码中的变量未按预期更新?

原因之一是 `.scratchpad` 段 和 `.bss_cla` 段 被错误地分配到只读 CLA 程序存储器。任务会运行，但分配到 `.bss_cla` 段 的变量不会被更新。此外，`.scratchpad` 段 被编译器使用，将其分配到只读内存可能导致未定义行为。

COMPARISON TO C28X+FPU

Note: This discussion is limited to the C28x + 32-bit FPU and does not include TMU or FPU64.

For this discussion, let us define the following instruction sets:

- **C28x Instruction Set** This is the original fixed-point instruction set.
- **C28x+FPU Instruction Set** This is the C28x Instruction Set plus additional instructions to support native single-precision (32-bit) floating-point operations. While the additional instructions are mostly to support single-precision floating-point math, there are some other useful instructions like RPTB (repeat block) included. Since they are part of the superset, and only available on devices with the FPU, we still refer to them as part of the FPU instructions.
- **CLA Instruction Set** The CLA instruction set is a subset of the FPU instructions. A few FPU instructions are not supported on CLA - for example the repeat block is not supported. The CLA also has a few instructions that the FPU does not have. For example: the CLA has some native integer math instructions as well as a native branch/call/return.

6.1 Are benchmarks for FPU div, sin, cos...etc. the same?

The CLA instructions are a subset of the C28x+FPU and for the math instructions there are equivalents on each, but there are still differences that impact benchmarks. For example:

- Cycle differences in multiply and conversion (see next question)
- Differences in branch and call instructions
- Resource differences (ex: 8 floating-point registers vs 4)
- Addressing modes

与 C28x+FPU 的比较

Note: 本讨论仅限于 C28x + 32-bit FPU，不包括 TMU 或 FPU64。

在本讨论中，我们将定义以下指令集：

- **C28xInstructionSet** 这是原始的定点指令集。
- **C28x+FPUInstruction Set** 这是 C28x Instruction Set，外加用于支持本地单精度（32 位）浮点运算的附加指令。虽然这些附加指令主要用于支持单精度浮点数学，但也包括一些其他有用的指令，例如 RPTB（重复块）。由于它们是超集的一部分，并且仅在带有 FPU 的器件上可用，我们仍将它们视为 FPU 指令的一部分。
- **CLA InstructionSet** CLA 指令集是 FPU 指令的一个子集。A 少数 FPU 指令在 CLA 上不支持——例如重述块（repeat block）不被支持。CLA 也有一些 FPU 所没有的指令。例如：CLA 具有一些本机整数运算指令（native integer math instructions），以及一个本机分支/调用/返回（native branch/call/return）。

6.1 用于 FPU 的 div、sin、cos 等的基准测试是否相同？

CLA 指令是 C28x+FPU 的一个子集，对于 math 指令每个都有等效的，但仍然存在影响基准测试的差异。例如：

- 乘法和转换的周期差异（见下一个问题）
- 分支和调用指令的差异
- 资源差异（例如：8 个浮点寄存器 vs 4）
- 寻址方式

- CLA lacks RPTB (repeat block)

6.2 Is the CLA floating-point multiply faster?

Consider the following:

- C28x 32-bit FPU:

Multiply or conversions take 2p cycles. This means that they take two cycles to complete, but remember you can put another instruction in that delay slot including another math instruction.
- CLA:

The math instructions and conversions take 1 cycle - no delay slot needed. So if you were not able to use that delay cycle on the FPU to do meaningful work, then you could say the CLA is faster if you are just counting cycles.

6.3 What are the main differences?

The following table summarizes some of the main differences between the C28x + FPU and the CLA CPU at this time. Refer to the CLA documentation listed in the other resources for the latest information.

Table 6.1: CLA and C28x+FPU Comparison

Item	CLA	C28x+FPU
Execution	Independent, parallel execution with the C28x CPU	Part of the main C28x CPU
Floating-Point Registers	4 (MR0 - MR3)	8 (R0H - R7H)
Auxillary Registers	2 16-bits, (MAR0, MAR1) – Can access all of CLA data	8, 32-bits, (XAR0 - XAR7) – Shared with fixed-point instructions
Pipeline	8-stage pipeline – completely independent from C28x	8-stage pipeline – CPU fetch and decode shared with the fixed-point instructions
Single Step	Moves pipeline ahead 1 cycle	Completely flushes the pipeline
Addressing Modes	2: Direct and indirect with post increment. No data page pointer.	All C28x addressing modes
Interrupt Sources	Device dependent. Interrupts come directly to the CLA.	All available interrupts through the PIE.

Continued on next page

- CLA 缺少 RPTB（重复块）

6.2 CLA 的浮点乘法更快吗？

考虑以下情况：

- C28x 32位 浮点单元（FPU）：

乘法或转换需要 2p 周期。这意味着它们需要两个周期完成，但请记住，你可以在该延迟槽中放入另一条指令，包括另一条数学指令。
- CLA：

数学指令和转换需要 1 个周期——不需要延迟槽。因此，如果你无法利用 FPU 的该延迟周期来完成有意义的工作，那么如果只是计算周期数的话，你可以说 CLA 更快。

6.3 主要差异是什么？

下表总结了当前 C28x +FPU 与 CLA CPU 之间的一些主要差异。有关最新信息，请参阅“其他资源”中列出的 CLA 文档。

表 6.1: CLA 与 C28x+浮点运算单元比较

Item	CLA	C28x+浮点运算单元
执行	独立并行执行与 the C28x CPU	作为主 C28x 中央处理器 的一部分
浮点寄存器	4 个 (MR0 - MR3)	8 个 (R0H - R7H)
辅助寄存器	2 个 16 位 (MAR0、MAR1) – 可访问所有 CLA 数据	8 个 32 位 (XAR0 - XAR7) – 与定点指令共享
流水线	8 级流水线 – 完全独立（不完整词）与 C28x 独立	8 级流水线 –CPU 取指和解码与定点指令共享
单步执行	使流水线前进 1 个周期	完全清空流水线
寻址方式	2：直接寻址和带后增量的间接寻址。不使用数据页指针。	所有 C28x 寻址模式
中断源	设备相关。中断直接到达 CLA。	通过 PIE（外设中断扩展模块）提供的所有可用中断。

续于下一页

Table 6.1 – continued from previous page

Item	CLA	C28x+FPU
Nesting Interrupts	No stack pointer. CLA type 0 & 1: not supported. CLA Type 2: supports 1 background task	Nesting enabled through software
Instruction Set	Independent subset of FPU instructions. Similar mnemonics to C28x+FPU but with leading ‘M’ ex: MMPYF32 MR0, MR1, MR2.	Floating-point instructions are in addition(superset) to the C28x fixed-point instructions.
Repeated Instructions	No single repeat or repeat block	Repeat MACF32 & repeat block (RPTB)
Communication with C28x	Through shared RAM, message RAM, and interrupts. C28x can read CLA registers.	One CPU. Can copy between fixed and float registers
Math and Conversion	Single cycle	2p cycles (2 pipelined cycles)
Integer Operations	Limited support. Native instructions for AND, OR, XOR, ADD, SUB, shifts etc..	Uses fixed-point instructions
Flow Control	Native branch/call/return conditional delayed. 3 instructions before/after branch are always executed - performance can be improved by using delay cycles	Uses fixed-point flow control. Branches are not delayed – Instructions after are only executed if the branch is not taken Requires copy of float flags to fixed-point ST0
Memory Access	CLA program, data and message RAMs only. Refer to the memory map in the data manual.	All memory on the device
Register Access	Refer to the specific datasheet or TRM.	All peripherals on the device or specific C28x sub-system
Programming	CLA Assembly or CLA C Compiler. Requires C28x codegen 6.1.0 or later.	C or C++ or Assembly
Operating Frequency	Refer to the device datasheet	Refer to the device datasheet

表 6.1 – 续前页

Item	CLA	C28x+浮点运算单元
嵌套中断	无堆栈指针。CLA 类型 0 & 1: 不支持。CLA 类型 2: supports 1 个后台任务	通过软件启用嵌套
指令集	FPU 指令的独立子集。与助记符相似 C28x+FPU 但以前缀 “M” 为例: MMPYF32 MR0, MR1, MR2.	浮点指令是对 C28x 定点指令的补充（超集）。
重复的指令	无单次重复或重复块	重复 MACF32 & 重复块 (RPTB)
通信与 C28x	通过共享 RAM, 随机存取存储器 (RAM), 与 (AND) 中断。C28x 可以读取 CLA 寄存器。	一个中央处理器 (CPU)。可以在固定之间复制和浮点寄存器
数学与 (AND) 版本	单周期	2p 周期 (2 个流水线周期)
整数 Operations	有限支持。本地指令用于与 (AND), OR, XOR, ADD, SUB, 移位等 ..	使用定点指令
流程控制	本地分支/调用 (指令) /返回 con- 附加延迟。3 条指令之前/之后分支 are 总是已执行 - 性能可以 be 通过使用延迟周期得到改善	Uses 定点 flow 控制。分支不会被延迟——之后的指令只有在分支被采用时才会执行 需要将浮点标志的副本复制到定点 ST0
内存访问	仅限 CLA 程序、数据和消息 RAM。请参阅数据手册中的存储映射。	设备上的所有内存
寄存器访问	请参阅相应的数据手册或 TRM。	器件上的所有外设或特定的 C28x 子系统
编程	CLA 汇编或 CLA C 编译器。需要 C28x codegen6.1.0 或 (OR) 之后 .	C 语言或 C++ 或汇编语言
运行频率	请参考设备数据表	请参考设备数据表

CHAPTER
SEVEN

SUPPORT

To submit feedback on this guide or for C2000 related questions, please post to the [C2000 micro-controllers forum](#) in the TI E2E™ support forums.

章节七

支持

若要就本指南提交反馈或针对 C2000 相关问题提问，请发布至 [C2000 微控制器 论坛](#) 的 TI E2ETM 支持论坛。

IMPORTANT NOTICE AND DISCLAIMER

TI PROVIDES TECHNICAL AND RELIABILITY DATA (INCLUDING DATASHEETS), DESIGN RESOURCES (INCLUDING REFERENCE DESIGNS), APPLICATION OR OTHER DESIGN ADVICE, WEB TOOLS, SAFETY INFORMATION, AND OTHER RESOURCES “AS IS” AND WITH ALL FAULTS, AND DISCLAIMS ALL WARRANTIES, EXPRESS AND IMPLIED, INCLUDING WITHOUT LIMITATION ANY IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE OR NON-INFRINGEMENT OF THIRD PARTY INTELLECTUAL PROPERTY RIGHTS.

These resources are intended for skilled developers designing with TI products. You are solely responsible for (1) selecting the appropriate TI products for your application, (2) designing, validating and testing your application, and (3) ensuring your application meets applicable standards, and any other safety, security, or other requirements. These resources are subject to change without notice. TI grants you permission to use these resources only for development of an application that uses the TI products described in the resource. Other reproduction and display of these resources is prohibited. No license is granted to any other TI intellectual property right or to any third party intellectual property right. TI disclaims responsibility for, and you will fully indemnify TI and its representatives against, any claims, damages, costs, losses, and liabilities arising out of your use of these resources.

TI’s products are provided subject to TI’s Terms of Sale (<https://www.ti.com/legal/termsofsale.html>) or other applicable terms available either on ti.com or provided in conjunction with such TI products. TI’s provision of these resources does not expand or otherwise alter TI’s applicable warranties or warranty disclaimers for TI products.

Mailing Address: Texas Instruments, Post Office Box 655303, Dallas, Texas 75265

重要通知和免责声明

TI 提供 技术与可靠性数据（包括 DATASHEETS）、设计资源（包括 Reference Design）、APPLICATION 或其他设计建议、WEB TOOLS、SAFETY INFORMATION 以及其他资源，按原样提供（AS IS）并含所有缺陷（ALL FAULTS），并否认所有保证（DISCLAIMS ALL WARRANTIES），包括明示与暗示的保证（EXPRESS AND IMPLIED），且不作任何限制，包括但不限于任何关于适销性、特定用途适用性（FITNESS FOR A PARTICULAR PURPOSE）或不侵害第三方知识产权（NON-INFRINGEMENT OF THIRD PARTY INTELLECTUAL PROPERTY RIGHTS）的暗示保证。

这些资源是为使用 TI 产品进行设计的熟练开发人员准备的。您需独自负责（1）为您的应用选择合适的 TI 产品，（2）设计、验证和测试您的应用，以及（3）确保您的应用符合适用标准和任何其他安全、保安或其他要求。这些资源如有更改，恕不另行通知。TI 授予您仅将这些资源用于开发使用资源中所述 TI 产品的应用的使用许可。禁止以其他方式复制与展示这些资源。未授予任何 TI 知识产权或任何第三方知识产权的许可。TI 对您使用这些资源所引起的任何索赔、损害、费用、损失及责任不承担责任，且您将对 TI 及其代表就此类事项予以完全赔偿。

TI’s 产品 are 提供 主题 to TI’s 条款 of Sale (<https://www.ti.com/legal/termsofsale.html>) 或者其他适用条款，可在 ti.com 上获得或与此类 TI products 一并提供。TI 提供这些 resources 并不扩展或以其他方式更改 TI products 的适用 warranties 或 warranty disclaimers。

邮寄地址：Texas Instruments，邮政信箱 655303，达拉斯，德克萨斯州 75265

CHANGELOG

Table 9.1: Version History

Version	Date	Summary
v1.0	April 2020	Initial version of guide.

更改日志

表9.1：版本历史

版本	Date	摘要
v1.0	2020年4月	指南的初始版本。